



به نام خداوند جان و خرد

نیمسال دوم ۴۰۴-۴۰۵
مدت امتحان: ۱۳۵ دقیقه

امتحان پایان ترم درس توسعه امن نرم افزار
تاریخ: ۱۴۰۴/۰۴/۰۶

دقت شود که پنج سؤال اول تستی است، برای این سؤالات گزینه درست را در پاسخنامه وارد کنید.

۱- با توجه به خروجی زیر، برنامه test در چه حالتی کامپایل شده است؟

```
(root@kali)-[/home/kali]
# nm -a test
nm: test: no symbols
```

الف) static و stripped

ب) static و not stripped

ج) dynamic و stripped

د) dynamic و not stripped

```
(root@kali)-[/home/kali]
# nm -D test
nm: test: no symbols
```

۲- کدام یک از گزینه‌های زیر درست است؟

الف) سرریزی بافر همیشه در پشته اتفاق می‌افتد.

ب) GDB به صورت پیش فرض ASLR را غیرفعال می‌کند.

ج) PIE یک گزینه کامپایلر است و عملکرد آن ارتباطی با مقدار ASLR ندارد.

د) strace یک ابزار تحلیل ایستا است.

۳- داده‌های محلی در کدام بخش از هدر (سرآیند) فایل PE ذخیره می‌شوند؟

الف) data. ب) text. ج) در هر دو بخش data و text. ممکن است ذخیره شوند. د) هیچ کدام

۴- کدام گزینه در مورد هدرهای LOAD در فایل‌های ELF درست است؟

الف) جزء Program Header هستند.

ب) کتابخانه‌هایی را تعریف می‌کنند که باید برای بارگذاری ELF در حافظه استفاده شود.

ج) برای resolve آدرس‌ها استفاده می‌شوند.

د) موارد الف و ب درست هستند.

۵- کدام گزینه در مورد قراردادهای فراخوانی در Linux x86 و Linux x64 نادرست است؟

الف) در Linux x86 آرگومان‌های ورودی تابع (هر تعدادی که باشند) در پشته ذخیره می‌شوند.

ب) در Linux x64 آرگومان‌های ورودی تابع (هر تعدادی که باشند) در رجیسترها ذخیره می‌شوند.

ج) مقدار برگشتی تابع در Linux x86 در رجیستر EAX و در Linux x64 در رجیستر RAX قرار می‌گیرد.

د) آدرس بازگشت همیشه قبل از متغیرهای محلی در پشته push می‌شود.

۶- تفاوت‌های اصلی جدول‌های نماد ایستا و پویای مربوط به فایل باینری test در دو حالت زیر را توضیح دهید.

```
(root@kali)-[/home/kali]
# gcc -static test.c -o test
```

حالت اول

```
(root@kali)-[/home/kali]
# gcc test.c -o test
```

حالت دوم

```
(root@kali)-[/home/kali]
# strip test
```



به نام خداوند جان و خرد

نیم سال دوم ۴۰۴-۴۰۵

امتحان پایان ترم درس توسعه امن نرم افزار

مدت امتحان: ۱۳۵ دقیقه

تاریخ: ۱۴۰۴/۰۴/۰۶

۷- با توجه به اطلاعات زیر، آدرس بازگشت چیست؟ به چه دلیل؟

Registers:

rsp: 0x30521dc0

rbp: 0x30521e00

Stack:

0x30521dc0:	0x00000000	0x00000000	0xffffe4e0	0x000007fff
0x30521dd0:	0x00400cff	0x00000000	0xd138f040	0x00000003f
0x30521de0:	0x00000011	0x00000000	0x0040010a	0x000007fff
0x30521df0:	0xffffe1e0	0x000007fff	0x00400ef5	0x00000003f
0x30521e00:	0x30521e80	0x00000000	0x004009a9	0x000000000
0x30521e10:	0xffffe1e8	0x0040010f	0x30521eb0	0x000000002
0x30521e20:	0x30521eb0	0x00000000	0x00400a12	0x000000000

۸- با دلیل توضیح دهید که آیا کد زیر می تواند منجر به آسیب پذیری Use After Free شود یا خیر.

```
void f(char *c_str1, size_t size) {
    char *c_str2 = (char *)realloc(c_str1, size);
    if (c_str2 == NULL) {
        free(c_str1);
    }
}

int main() {
    char *c = malloc(sizeof(char));
    int n;
    scanf("%d", &n);
    f(c, n);
}
```

۹- قطعاتی از دو برنامه ی پک شده در شکل های زیر نشان داده شده است. در هر مورد با دلیل مشخص کنید که آیا دستوری برای پرش به ابتدای Entry Point اصلی برنامه وجود دارد یا خیر. در صورت وجود دستور مورد نظر را نیز مشخص کنید.

الف)	00407550	50	PUSH EAX
	00407551	68 77154000	PUSH test4.00401577
	00407556	C2 0400	RETN
	00407559	8BB5 5B974000	MOV ESI,DWORD PTR SS:[EBP+40975B]
	0040755F	0BF6	OR ESI,ESI
	00407561	74 18	JE SHORT test4.0040757B
	00407563	8B95 E6904000	MOV EDX,DWORD PTR SS:[EBP+4090E6]
	00407569	03F2	ADD ESI,EDX
	0040756B	E8 0F000000	CALL test4.0040757F
	00407570	72 0B	JB SHORT test4.0040757D

ب)	004050DA	FF37	PUSH DWORD PTR DS:[EDI]
	004050DC	AF	SCAS DWORD PTR ES:[EDI]
	004050DD	EB 09	JMP SHORT test1.004050E8
	004050DF	FE0F	DEC BYTE PTR DS:[EDI]
	004050E1	0F84 A9BFFFFFF	JE test1.00401090
	004050E7	57	PUSH EDI
	004050E8	55	PUSH EBP
	004050E9	FF53 04	CALL DWORD PTR DS:[EBX+4]



به نام خداوند جان و خرد

نیمسال دوم ۴۰۴-۴۰۵
مدت امتحان: ۱۳۵ دقیقه

امتحان پایان ترم درس توسعه امن نرم افزار
تاریخ: ۱۴۰۴/۰۴/۰۶

۱۰- در صورتی که از Breakpoint های سخت افزاری برای دیباگ برنامه استفاده شود آیا روش INT Scanning روش مناسبی برای تشخیص Debugger است؟ چرا؟

۱۱- شکل زیر بخشی از محتوای پشته مربوط به تابع main یک برنامه اجرایی پس از وارد کردن رشته "aaaa" توسط کاربر را نشان می دهد (کد اسکی a برابر 0x61 است). این برنامه با فعال بودن Canary در سیستم ۶۴ بیتی کامپایل شده است. با دلیل بیان کنید که حداقل طول رشته ورودی چقدر باید باشد تا برنامه با خطای زمان اجرا متوقف شود؟ در این حالت خطای زمان اجرا از چه نوعی است؟

```
0x7fffffff200: 0x0000000006161616 0x0000000000000000
0x7fffffff210: 0x0000000000000000 0x00007ffff7fe6780
0x7fffffff220: 0x0000000000000000 0x703b7b91b4234900
0x7fffffff230: 0x0000000000000001 0x00007ffff7df16ca
0x7fffffff240: 0x00007fffffe330 0x00000000000401136
```

۱۲- آیا در هر یک از قطعه کدهای زیر، از تکنیکی برای Anti-Disassembly استفاده شده است؟ در صورت مثبت بودن پاسخ تکنیک مورد استفاده را نام برده و عملکرد آن را روی کد توضیح دهید.

الف)	.text:004011C3 8B 8D 64 FD FF FF	mov ecx, [ebp-29Ch]
	.text:004011C9 51	push ecx
	.text:004011CA FF 15 60 20 40 00	call ds:InternetCloseHandle
	.text:004011D0 33 C0	xor eax, eax
	.text:004011D2 74 01	jz short near ptr loc_4011D4+1
	.text:004011D4 E8 68 30 30 40	call near ptr 40704241h
	.text:004011D9 00 8D 95 60 FD FE	add [ebp-1029F6Bh], cl
	.text:004011DF FF 52 FF	call dword ptr [edx-1]
	.text:004011E2 15 2C 20 40 00	adc eax, offset strstr
ب)	.text:004012E6 66 B8 EB 05	mov ax, 5EBh
	.text:004012EA 31 C0	xor eax, eax
	.text:004012EC 74 FA	jz short near ptr loc_4012E6+2
	.text:004012EE E8 6A 0A 6A 00	call near ptr 0AA1D5Dh
	.text:004012F3 6A 00	push 0
	.text:004012F5 8B 85 58 FD FE FF	mov eax, [ebp-102A8h]
	.text:004012FB 50	push eax
	.text:004012FC 6A 00	push 0
	.text:004012FE 6A 00	push 0
	.text:00401300 FF 15 54 20 40 00	call ds:ShellExecuteA

۱۳- در شکل زیر بخش قرار گرفته داخل کادر ورودی وارد شده توسط کاربر و دو خط بعدی خروجی نمایش داده شده توسط برنامه است.

```
(root@kali)-[/home/kali]
# ./exam_fs
12345678.%x.%x.%x.%x.%x.%x.%x.%x.%x.%x.%x.%x.%x.%x.%x.%x
This is a test: 12345678.56570008.ff8bf160.5656f1c7.f63d4e2e.3055e4.f7f6f5d8.736968
54.20736920.65742061.203a7473.34333231.38373635.2e78252e.252e7825.78252e78.2e78252e
```

الف) پیلودی بنویسید که محتویات آدرس فرضی 0x312a0408 را نمایش دهد.

ب) پیلودی بنویسید که محتویات آدرس فرضی 0x312a0408 را برابر با عدد ۷۰۰۰۰ (0x11170) قرار دهد.



به نام خداوند جان و خرد

نیم سال دوم ۴۰۴-۴۰۵

امتحان پایان ترم درس توسعه امن نرم افزار

مدت امتحان: ۱۳۵ دقیقه

تاریخ: ۱۴۰۴/۰۴/۰۶

۱۴- با در نظر گرفتن کد مربوط به توابع test, main و flag که در شکل های زیر نشان داده شده است:

الف) پیلودی بنویسید که بدون نیاز به بازنویسی آدرس بازگشت، محتوای فایل flag.txt را نمایش دهد.

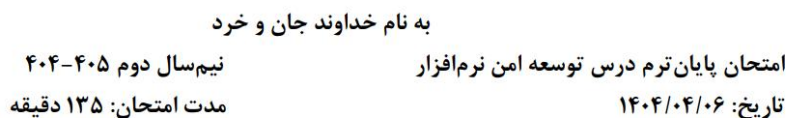
ب) فرض کنید مهاجم قصد گرفتن Shell با بهره برداری از آسیب پذیری سرریزی بافر در این برنامه را دارد. با در نظر گرفتن اطلاعات زیر، پیلود ROP مناسب برای این کار را بنویسید.

%rax	System call	%rdi	%rsi	%rdx
59	sys_execve	const char *filename	const char *const argv[]	const char *const envp[]

- 0x00000000004175eb : pop rax ; ret
- 0x000000000040191a : pop rdi ; ret
- 0x000000000040f4be : pop rsi ; ret
- 0x000000000040247d : pop rdx ; ret
- 0x00000000004012d3 : syscall

- همچنین فرض کنید رشته "/bin/sh" در آدرس 0x00000000004027a8 نوشته شده است.

*****				1
* FUNCTION				2
*****				3
undefined main()				4
undefined	AL:1	<RETURN>		5
undefined4	Stack[-0xc]:4	local_c		6
undefined1	Stack[-0x28]:1	local_28		7
main				8
				9
00401205	55	PUSH	RBP	10
00401206	48 89 e5	MOV	RBP,RSP	11
00401209	48 83 ec 20	SUB	RSP,0x20	12
0040120d	c7 45 fc	MOV	dword ptr [RBP + loc_45fc],0	13
	00 00 00 00			14
00401214	48 8d 05	LEA	RAX,[s_Enter_your_name]	15
	0d 0e 00 00			16
0040121b	48 89 c7	MOV	RDI=>s_Enter_your_name	
0040121e	e8 0d fe	CALL	<EXTERNAL>::puts	
	ff ff			
00401223	48 8d 45 e0	LEA	RAX=>local_28,[RBP + 0x45e0]	
00401227	48 89 c7	MOV	RDI,RAX	
0040122a	b8 00 00	MOV	EAX,0x0	
	00 00			
0040122f	e8 2c fe	CALL	<EXTERNAL>::gets	
	ff ff			



```

    *                                     THUNK FUNCTION
    *****
    thunk undefined frame_dummy()
    Thunked-Function: register_tm_clones

undefined      AL:1          <RETURN>
               frame_dummy
00401160 f3 0f 1e fa      ENDBR64
00401164 eb 8a          JMP      register_tm_clones
               -- Flow Override: CALL_RETURN (CALL

*****
*                                     FUNCTION
*****

undefined      AL:1          <RETURN>
undefined8      Stack[-0x10]:0 local_10

undefined1      Stack[-0x28]:1 local_28

               flag

00401166 55          PUSH     RBP
00401167 48 89 e5      MOV      RBP,RSP
0040116a 48 83 ec 20     SUB      RSP,0x20
0040116e 48 8d 05      LEA      RAX,[DAT_00402004]
               8f 0e 00 00

```

موفق باشید